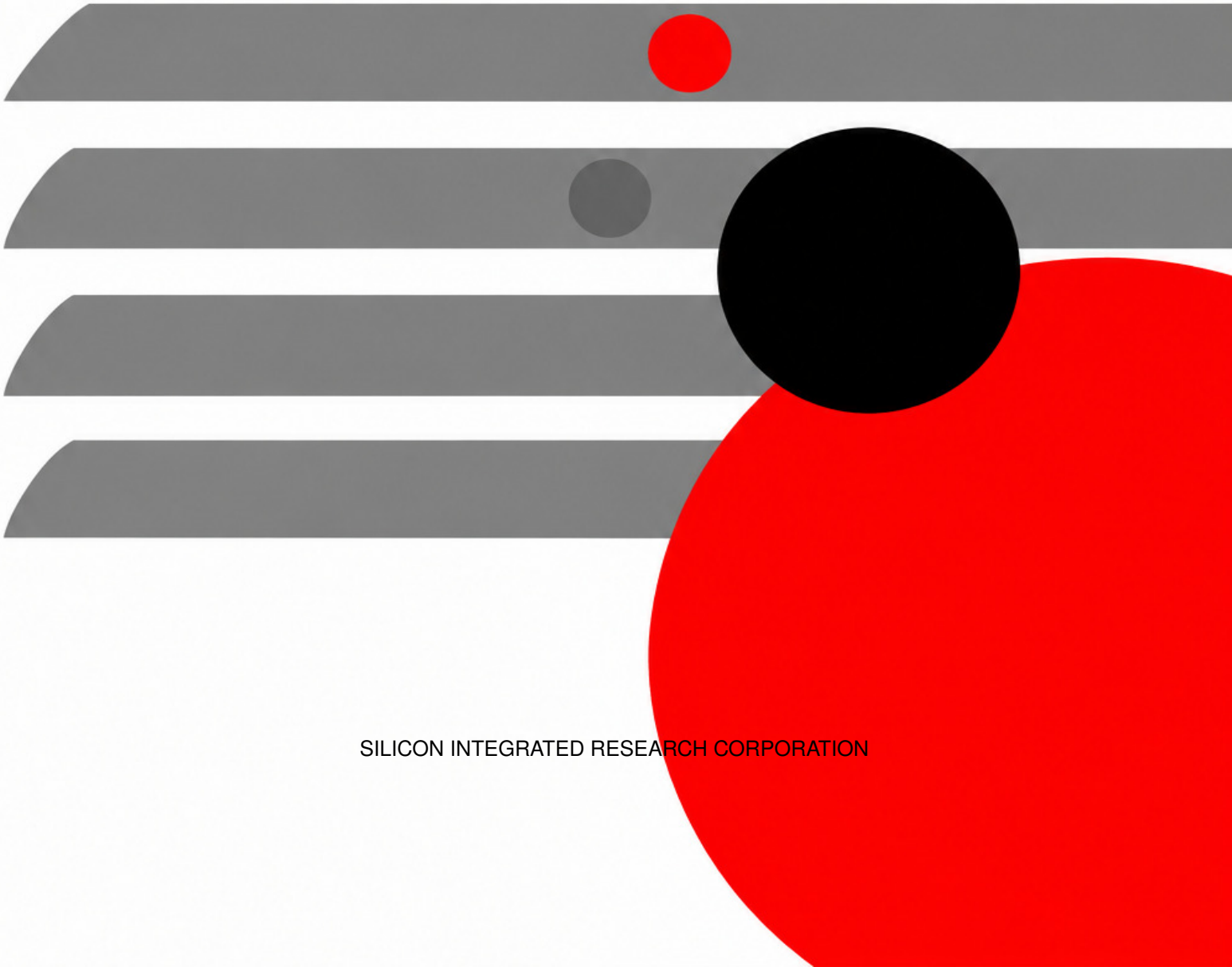


SIRC-1

Hardware Reference Manual

Package, Signals and Bus Interface



SILICON INTEGRATED RESEARCH CORPORATION

Preface

About This Manual

This manual documents the SIRC-1 CPU as a hardware component: its package, its signals, and the bus protocol it uses to talk to memory and peripherals. It is the companion to the *SIRC-1 Architecture Reference Manual*, which defines the instruction set, registers, exceptions and other programmer-visible behaviour. Everything in this manual is about what happens on the pins; the architecture manual is about what the processor does.

It is written for:

- Board and system designers
- Memory controller and peripheral designers
- Hardware implementers building a compatible device

Assumed Knowledge

This manual assumes familiarity with digital logic, synchronous bus protocols, and active-low signalling. Readers who also need to know how the processor responds to resets, interrupts and bus faults should read the corresponding chapters of the *SIRC-1 Architecture Reference Manual*.

Revision Status

This is Version 1.0 of the SIRC-1 Hardware Reference Manual, issued in draft for internal review and not for external distribution. Chapter 6 lists the electrical characteristics that have not yet been specified.

Contents

Preface iii

1 Introduction 1
- 1.1 Purpose and Scope . . . 1
- 1.2 Relationship to the Architecture Manual . . . 1
- 1.3 Signal Groups . . . 1

2 Package and Pinout 3
- 2.1 Chip Layout . . . 3

3 Signal Descriptions 5
- 3.1 Signal Conventions . . . 5
- 3.2 Address Bus Bits (A0–A23) . . . 5
- 3.3 Power Pins (VCC1–2, GND1–2) . . . 5
- 3.4 Data Bus Bits (D0–D15) . . . 5
- 3.5 Clock Input (CLKI) . . . 5
- 3.6 Instruction Sync Output (SYNC) . . . 6
- 3.7 Bus Access Strobe Output (BAS) . . . 6
- 3.8 Bus Read/Write Output (BRW) . . . 6
- 3.9 Bus Acknowledge Input (BACK) . . . 6
- 3.10 Bus Error Input (BERR) . . . 6
- 3.11 Bus Protection Error Input (BPER) . . . 7
- 3.12 Bus Access Type Bits (BAT0–BAT2) . . . 7
- 3.13 Protected-Mode Output (PROT) . . . 7
- 3.14 External CPU Halt Input (HALT) . . . 8
- 3.15 Force Trace Mode Input (TRCE) . . . 8
- 3.16 Maskable Interrupt Inputs (IRQ1–IRQ4) . . . 8
- 3.17 Non-Maskable Interrupt Inputs (NMI) . . . 8
- 3.18 Reset Input (RSTI) . . . 9
- 3.19 Reset Output (RSTO) . . . 9

4 Bus Interface 11
- 4.1 Clocked Bus Model . . . 11
- 4.2 Bus Transaction Completion . . . 11
- 4.3 Bus Cycle Types . . . 12
- 4.4 Control Input Sampling . . . 12

5 Bus Timing Diagrams 13
- 5.1 Bus Cycle Diagrams . . . 13
- 5.2 Reset and Interrupt Timing Examples . . . 16

6 Electrical Characteristics 17

List of Tables

1.1 The SIRC manual set 1
1.2 SIRC-1 signal groups 1

2.1 SIRC CPU 64-pin DIP pinout 4

3.1 Bus Access Type Bits Meaning 7

4.1 CPU bus output timing rules 11
4.2 Bus response timing rules 11
4.3 External bus cycle ownership 12
4.4 External input sampling rules 12

List of Figures

2.1 SIRC CPU 64-pin DIP pinout, top view 3

5.1 Instruction-fetch high-word and low-word reads with no wait states 13
5.2 Data read cycle with no wait state 13
5.3 Data write cycle with no wait state 14
5.4 Exception-vector fetch with high-word then low-word reads 14
5.5 Wait-state insertion when no bus response is asserted 14
5.6 Bus-error and bus-protection abort timing 15
5.7 Complete register-only ALU instruction with no wait states 15
5.8 Complete LOAD instruction with no wait states 15
5.9 Hardware reset with one-cycle RSTI assertion and reset-vector fetch 16
5.10 Enabled IRQ latched before an instruction boundary and dispatched with no wait states 16

CHAPTER 1

Introduction

1.1 Purpose and Scope

The SIRC-1 documentation is split into three manuals, each with its own scope:

Manual	Scope
SIRC Architecture Reference Manual	ISA, instruction encodings, canonical assembly notation, exceptions, programmer-visible state
SIRC Assembly Programmer's Manual	Complete assembly syntax, directives, macros, pseudoinstructions
SIRC Hardware Reference Manual	Pinout, bus interface, timing, electrical characteristics (this manual)

Table 1.1: The SIRC manual set

This manual covers the concrete silicon: the 64-pin package, what each signal does, the bus protocol, and the timing of bus cycles, resets and interrupts. It does not describe instructions, registers or exception handling from the programmer's point of view.

1.2 Relationship to the Architecture Manual

The *SIRC-1 Architecture Reference Manual* defines what the CPU does: which instructions exist, how exceptions are raised and returned from, and what the programmer can observe. This manual defines how that behaviour appears at the pins. Several behaviours are visible in both places, for example the reset sequence and the bus fault retry rules. Where they overlap, the architecture manual is authoritative for what happens inside the CPU and this manual is authoritative for what is driven or sampled on each signal.

1.3 Signal Groups

The 64 pins fall into the groups below. Chapter 2 gives the pin numbers and Chapter 3 describes every signal.

Group	Signals	Pins
Address bus	A0–A23	24
Data bus	D0–D15	16
Bus control	BAS, BRW, BAT0–BAT2, BACK, BERR, BPER, PROT, SYNC	10
Interrupts and execution control	IRQ1–IRQ4, NMI, HALT, TRCE	7
Reset	RSTI, RSTO	2
Clock	CLKI	1
Power	VCC1–VCC2, GND1–GND2	4

Table 1.2: SIRC-1 signal groups

CHAPTER 2

Package and Pinout

2.1 Chip Layout

The SIRC CPU is most commonly provided as a DIP-64 package.

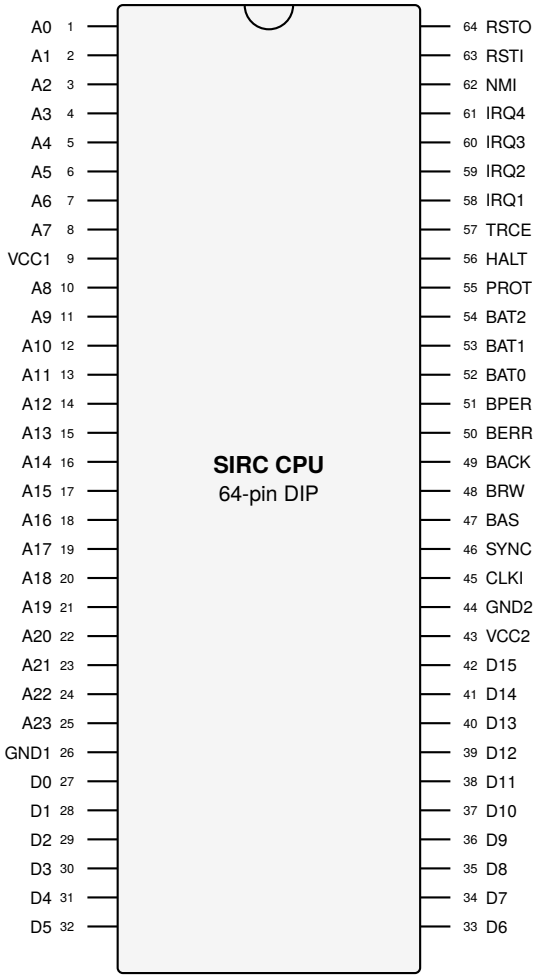


Figure 2.1: SIRC CPU 64-pin DIP pinout, top view

Pin	Name	Description
1–8	A0–A7	Address bus bits 0–7
9	VCC1	Power supply
10–25	A8–A23	Address bus bits 8–23
26	GND1	Ground
27–42	D0–D15	Data bus bits 0–15
43	VCC2	Power supply
44	GND2	Ground
45	CLKI	Clock input
46	SYNC	Instruction-sync output
47	BAS	Bus access strobe output
48	BRW	Bus read/write output
49	BACK	Bus acknowledge input
50	BERR	Bus error input
51	BPER	Bus protection error
52–54	BAT0–BAT2	Bus access type bits 0–2
55	PROT	Protected-mode output
56	HALT	External CPU halt input
57	TRCE	Force trace mode input
58–61	IRQ1–IRQ4	Maskable interrupt inputs 1–4
62	NMI	Non-maskable interrupt input
63	RSTI	Reset input
64	RSTO	Reset output

Table 2.1: SIRC CPU 64-pin DIP pinout

CHAPTER 3

Signal Descriptions

3.1 Signal Conventions

Each signal lists its own assertion convention. Address, data, and bus-type value pins use positive logic, while bus control, handshake, interrupt, halt, trace, and reset pins generally use active-low conventions. The descriptions below use "asserted" and "deasserted" to refer to the logical state of a signal according to its signal type.

3.2 Address Bus Bits (A0–A23)

- **Direction:** Output
- **Signal Type:** Active High

Connected to the address bus.

These 24 pins declare the address on the bus that the CPU reads from or writes to when BAS (bus access strobe) is asserted.

3.3 Power Pins (VCC1–2, GND1–2)

These pins provide the 5V power that the CPU requires to function.

The VCC pins must be connected to 5V +/- 5% and GND pins must be connected to a 0V or ground rail.

3.4 Data Bus Bits (D0–D15)

- **Direction:** Bi-directional
- **Signal Type:** Active High

Connected to the data bus.

These 16 pins are driven to a 16-bit value when the CPU is outputting to the bus, and are an input for a 16-bit value when the CPU is reading from the bus.

3.5 Clock Input (CLKI)

- **Direction:** Input
- **Signal Type:** Rising Edge
- **Max Rate:** 24 MHz

Progresses the CPU to the next cycle when the input transitions from low to high (rising edge).

Each transition from low to high advances the CPU's internal six-stage sequence by one stage, so every six clock transitions complete one full instruction execution when no bus wait states occur.

3.6 Instruction Sync Output (SYNC)

- **Direction:** Output
- **Signal Type:** Active High

Asserted while the CPU is in the first execution phase of an instruction or exception-unit dispatch. It remains asserted if a wait state holds that phase, and is deasserted during every other phase.

This pin is useful for debugging purposes.

3.7 Bus Access Strobe Output (BAS)

- **Direction:** Output
- **Signal Type:** Active Low

When the CPU is ready to perform a bus operation, BAS is asserted after the address pins, BRW, BAT, PROT, and any CPU-driven data pins are valid.

BAS remains asserted until the bus responds with BACK, BERR, or BPER. If no response is asserted, the CPU holds the current execution phase and repeats the same bus request on the following clock.

3.8 Bus Read/Write Output (BRW)

- **Direction:** Output
- **Signal Type:** Read High / Write Low

Only relevant when the BAS (Bus access strobe output) is asserted.

When high, the CPU is requesting a read from the bus. When low, the CPU is requesting a write to the bus.

3.9 Bus Acknowledge Input (BACK)

- **Direction:** Input
- **Signal Type:** Active Low

When asserted, the bus is reporting that the requested read/write completed successfully. On reads, the CPU samples D0–D15 on the clock edge that accepts BACK.

If all bus devices are fast and can complete reads/writes in a single cycle, this pin can be permanently held asserted.

3.10 Bus Error Input (BERR)

- **Direction:** Input
- **Signal Type:** Active Low

When asserted, the bus is reporting that the requested read/write was unsuccessful.

This causes the CPU to abort the I/O operation and raise a bus fault exception.

An example of a situation where a bus error might be raised would be I/O to an address that is not connected to any device.

3.11 Bus Protection Error Input (BPER)

- **Direction:** Input
- **Signal Type:** Active Low

When asserted, the bus is reporting that the requested read/write was to a valid address/device, but that the I/O is disallowed given the current state of the CPU or bus.

This causes the CPU to abort the I/O operation and raise a bus protection fault exception.

It behaves much like the BERR pin but raises a different fault so that programs can handle it differently.

An example of a situation where a bus protection error might be raised would be a write to ROM, or a write to a privileged section of memory when the CPU is not in supervisor mode.

3.12 Bus Access Type Bits (BAT0–BAT2)

- **Direction:** Output
- **Signal Type:** Active High

Output by the CPU to describe what I/O operation it is performing.

Can be used to provide useful hints to external devices during I/O.

For example, if a memory controller knows that the DMA coprocessor will be reading more than one sequential address, it can prefetch the next address.

Another example might be a memory controller providing memory protection if these pins are used in conjunction with the PROT (Protected-mode output) pin. A memory controller might trigger a bus error if BAT is 010 (data read), PROT is 1 (protected mode) and the address is in a protected memory region.

BAT2	BAT1	BAT0	Description
0	0	0	No I/O
0	0	1	Instruction Fetch
0	1	0	Data Read
0	1	1	Data Write
1	0	0	Exception Vector Fetch
1	0	1	DMA Coprocessor Read Burst
1	1	0	DMA Coprocessor Write Burst
1	1	1	Reserved

Table 3.1: Bus Access Type Bits Meaning

3.13 Protected-Mode Output (PROT)

- **Direction:** Output
- **Signal Type:** Active High

Indicates whether the CPU is in protected mode (high) or in supervisor mode (low).

Follows the status register Protected Mode (P) bit, except that it is deasserted for every exception-vector fetch (BAT 100), which is a supervisor access even when the exception is taken from protected mode.

When the CPU is in protected mode, programs have access to fewer types of instructions, for example they cannot write to the high word of the address registers.

Exposing this as an external pin allows other devices such as memory controllers to block access to protected regions of memory by raising bus protection errors.

3.14 External CPU Halt Input (HALT)

- **Direction:** Input
- **Signal Type:** Active Low

When asserted, the CPU stops executing instructions.

HALT is sampled at instruction boundaries. If asserted, the CPU completes the current instruction, including the dispatch slot of a coprocessor call, and halts before the next instruction fetch. It performs no bus cycles while halted. The interrupt lines are still sampled and a Level 5 edge is still latched, but nothing is dispatched until the CPU resumes. It resumes as normal when the input is deasserted. A reset ends the halt, but if HALT is still asserted the CPU halts again at the first instruction boundary after reset.

This pin is useful for hardware-driven debugging, as the CPU can be halted and the state of the system examined.

3.15 Force Trace Mode Input (TRCE)

- **Direction:** Input
- **Signal Type:** Active Low

When asserted at an instruction boundary, the instruction being started is traced regardless of whether software has cleared the Trace Mode (T) flag in the status register. No trace fault is raised while a fault handler runs (exception level 7), since it would escalate to a double fault.

When in trace mode, the CPU raises an exception on every instruction to allow software debugging.

3.16 Maskable Interrupt Inputs (IRQ1–IRQ4)

- **Direction:** Input
- **Signal Type:** Active Low

When asserted, the corresponding hardware interrupt line is sampled by the CPU.

They can be masked by the Hardware Interrupt Enable Bits (E1–E4) of the status register.

Interrupt inputs are level-sensitive and are sampled on every clock cycle. Enabled interrupt lines are latched in the pending hardware-exceptions register; disabled maskable lines are ignored and not queued, and a line is not latched while its own handler or a higher-priority one is running. The requesting device must hold the line asserted until its handler acknowledges it; an assertion withdrawn before it is latched is not recognized. If an interrupt pin remains asserted after its handler returns, it may be sampled again through the normal interrupt rules.

See the Exception and Interrupt Handling chapter of the *SIRC-1 Architecture Reference Manual* for more details.

3.17 Non-Maskable Interrupt Inputs (NMI)

- **Direction:** Input
- **Signal Type:** Active Low

When asserted, the CPU samples the Level 5 hardware exception line. Unlike `IRQ1`–`IRQ4`, which are level-sensitive, NMI is edge-triggered: it fires once on assertion regardless of how long the pin is held, and the pin must be released and reasserted to fire again.

This interrupt cannot be masked. If this pin is asserted while another Level 5 exception is being handled, the CPU raises a Level Five Hardware Exception Conflict fault.

Because NMI is edge-triggered, holding the pin asserted after its handler returns does not by itself cause it to fire again – a fresh falling-then-rising transition is required. Level 5 exception handlers must therefore execute quickly, and NMI should be asserted only for important, non-interruptible events.

3.18 Reset Input (RSTI)

- **Direction:** Input
- **Signal Type:** Active Low

Can be used by external devices to force a reset of the CPU.

When asserted, the CPU immediately aborts the current instruction or bus wait, stops normal processing, resets to phase 0, and begins the reset sequence described in the Exception and Interrupt Handling chapter of the *SIRC-1 Architecture Reference Manual*.

The RSTO pin is also asserted.

When this pin is deasserted, the CPU completes the reset-output hold, stops asserting the RSTO pin, and the exception unit fetches the reset vector.

If a system needs external devices to be reset for longer than the CPU reset-output hold, reset timing must be managed externally to the CPU, and RSTI must remain asserted for as long as the other devices need to reset.

3.19 Reset Output (RSTO)

- **Direction:** Output
- **Signal Type:** Active Low

Asserted while the RSTI pin is asserted and during the reset-output hold before reset-vector fetch.

It is also asserted for the reset-output hold after a software reset, and for a further hold before the CPU retries a reset-vector fetch that failed.

A reset can be triggered by asserting the Reset input (RSTI) pin, or in software by a special exception coprocessor meta-instruction (RSET).

This pin is useful to reset other devices on the bus whenever the CPU is reset or to trigger more complex reset/boot controllers.

CHAPTER 4

Bus Interface

4.1 Clocked Bus Model

All bus timing is described relative to the rising edge of CLKI. The CPU presents a bus request during the execution phase that needs external access. If no response is asserted, the CPU repeats the same request and does not advance to the next execution phase.

A no-wait-state bus access assumes that the selected external device can assert BACK and, for reads, drive valid data during the same CLKI interval in which BAS is asserted, so the CPU accepts the transfer on the next rising edge. A device that samples the request on one clock edge and cannot return BACK or read data until a later edge adds wait states; this is the normal way to attach slower synchronous memory or peripherals.

Signal	Timing rule
A0–A23	Valid whenever BAS is asserted. Stable until the request completes or aborts.
D0–D15	Driven by the CPU only for writes. On reads, the selected device drives the bus and the CPU samples D0–D15 when BACK is accepted.
BRW	Valid whenever BAS is asserted. High requests a read; low requests a write. Stable until the request completes or aborts.
BAT0–BAT2	Valid whenever BAS is asserted. Encodes the access type for the active request and is stable until completion or abort.
PROT	Reflects the status register P bit, except that it is deasserted for exception-vector fetches, and may be used with BAS, BAT, and A0–A23 by external protection logic.
BAS	Asserted while a bus request is active. Remains asserted across wait states.
SYNC	Asserted while phase 0 (Instruction Fetch, high word) of an instruction or exception-unit dispatch is active.

Table 4.1: CPU bus output timing rules

When BAS is deasserted there is no active bus transaction. External devices must ignore A0–A23, D0–D15, BRW, and BAT0–BAT2 unless BAS is asserted.

4.2 Bus Transaction Completion

Response	Effect when sampled while BAS is asserted
BACK	Completes the request successfully. For reads, D0–D15 is sampled on the accepting clock edge. For writes, the write is considered externally committed.
BERR	Aborts the request and raises a bus fault. Read data is ignored and writes are not considered successfully committed by the CPU.
BPER	Aborts the request and raises a bus protection fault. Read data is ignored and writes are not considered successfully committed by the CPU.
No response	Inserts a wait state. The CPU remains in the current execution phase and keeps A0–A23, D0–D15 for writes, BRW, BAT0–BAT2, PROT, and BAS stable.

Table 4.2: Bus response timing rules

A bus transaction terminated by Bus Error or Bus Protection Error is architecturally unsuccessful. An external device must not commit any externally visible effect from a transaction for which it asserts the corresponding error signal: for a write, the device must not act on the data driven to it; for a read, any side effect the access would otherwise have (for example, a status register that clears on read) must not occur either. This is what makes Bus Fault and Bus Protection Fault safe to retry (see the Exception and Interrupt Handling chapter of the *SIRC-1 Architecture Reference Manual*) – the CPU can assume that a transaction it aborted with one of these signals never happened.

BACK, BERR, and BPER are logically mutually exclusive. If more than one is asserted for the same request, BERR has priority, then BPER, then BACK.

4.3 Bus Cycle Types

Cycle	CPU drives	External device drives	Completion
Read	A0–A23, BRW=1, BAT, BAS, PROT	D0–D15 for the selected address	BACK samples data; BERR/BPER aborts.
Write	A0–A23, D0–D15, BRW=0, BAT, BAS, PROT	No data bus drive	BACK accepts write; BERR/BPER aborts.
Wait state	Same values as the previous request cycle	Same ownership as the active request	Continues until BACK, BERR, or BPER is asserted.
No bus cycle	No active transaction	No device drives D0–D15	CPU advances internally.

Table 4.3: External bus cycle ownership

4.4 Control Input Sampling

Input	Sampling rule
BACK, BERR, BPER	Sampled while BAS is asserted. They complete or abort the active bus request.
IRQ1–IRQ4	Level-sensitive and sampled every cycle. The requesting device must hold the line asserted until its handler acknowledges it; a line is not latched while its own or a higher-priority handler runs.
NMI	Edge-triggered. Fires once on assertion regardless of how long the pin is held; the pin must be released and reasserted to fire again.
HALT	Sampled at instruction boundaries. If asserted, the CPU halts before the next instruction fetch and resumes when deasserted; interrupt lines are still sampled while halted.
TRCE	Sampled at instruction boundaries. If asserted, the instruction being started is traced, except inside a fault handler.
RSTI	May be asserted at any time. It aborts the current instruction, exception dispatch, or bus wait and starts the reset sequence.

Table 4.4: External input sampling rules

CHAPTER 5

Bus Timing Diagrams

5.1 Bus Cycle Diagrams

The timing diagrams below show cycle-level logical timing. Each labelled interval represents one CLKI cycle as observed between rising edges. Waveform rows show physical logic levels; for active-low signals, `L` means asserted and `H` means deasserted. Address, data, `BAT`, and phase rows are value bands rather than electrical setup/hold specifications. The diagrams use `IF` for instruction fetch, `DR` for data read, `DW` for data write, `EV` for exception vector fetch, `EA` for effective address, and `RV` for reset vector.

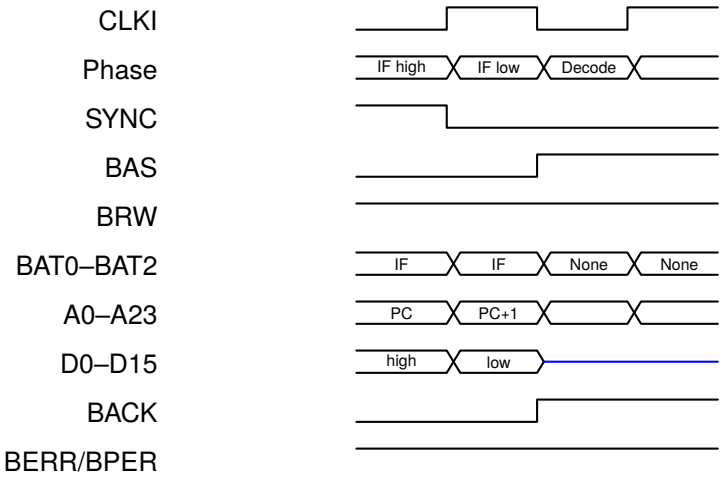


Figure 5.1: Instruction-fetch high-word and low-word reads with no wait states

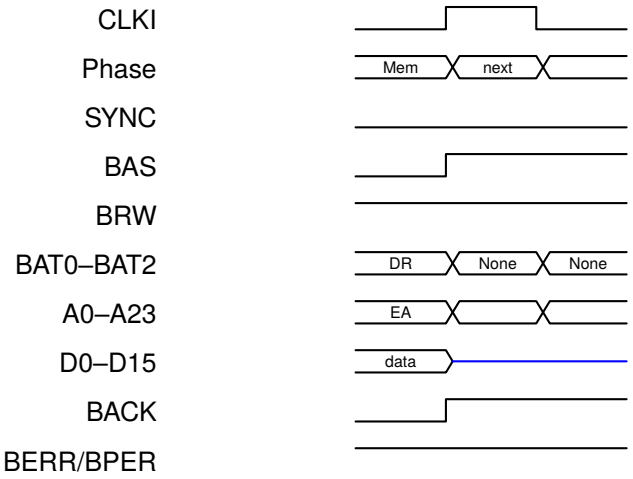


Figure 5.2: Data read cycle with no wait state

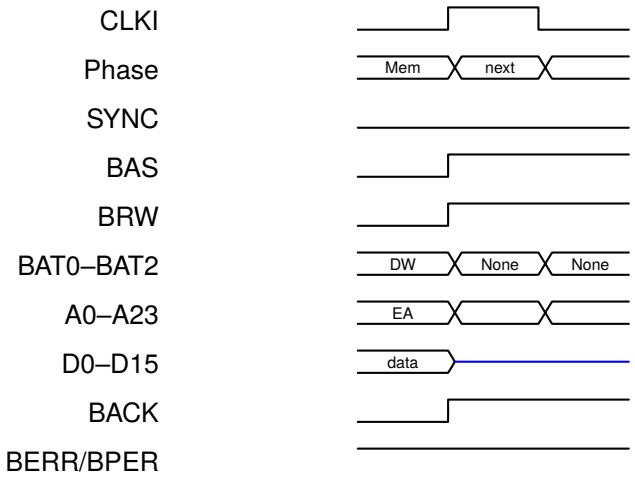


Figure 5.3: Data write cycle with no wait state

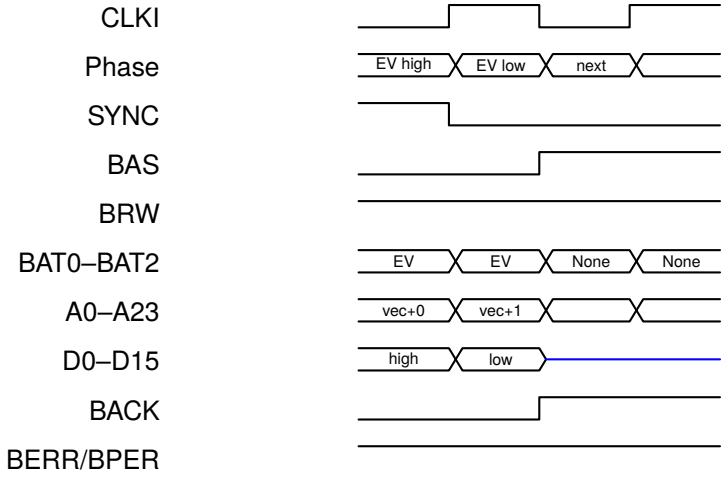


Figure 5.4: Exception-vector fetch with high-word then low-word reads

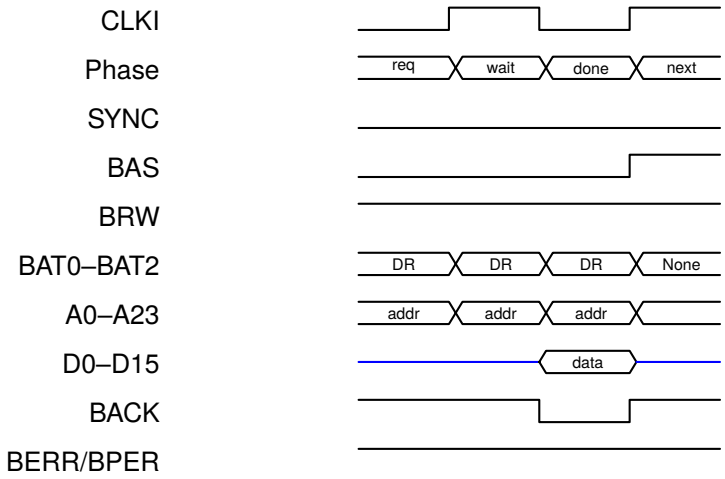


Figure 5.5: Wait-state insertion when no bus response is asserted

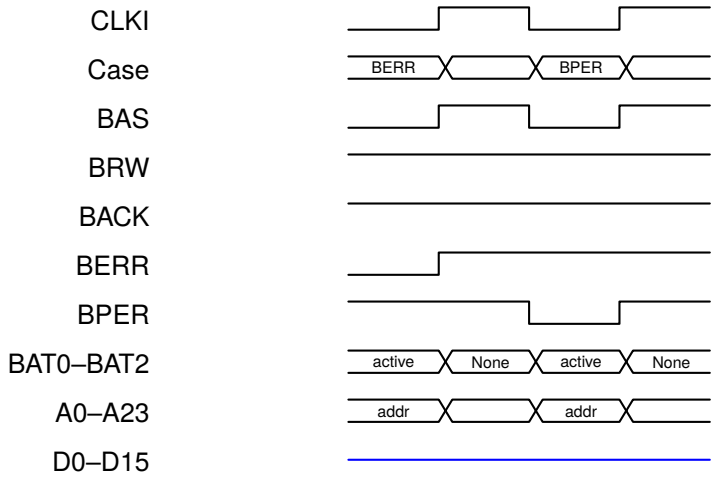


Figure 5.6: Bus-error and bus-protection abort timing

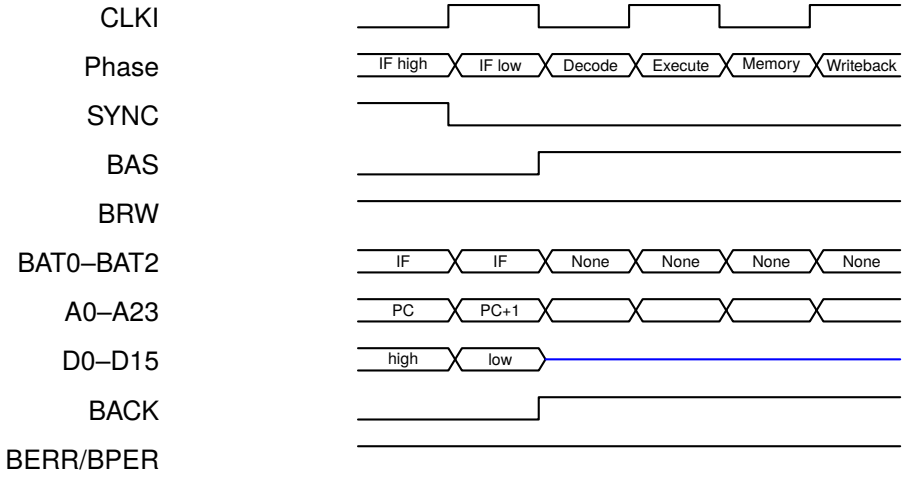


Figure 5.7: Complete register-only ALU instruction with no wait states

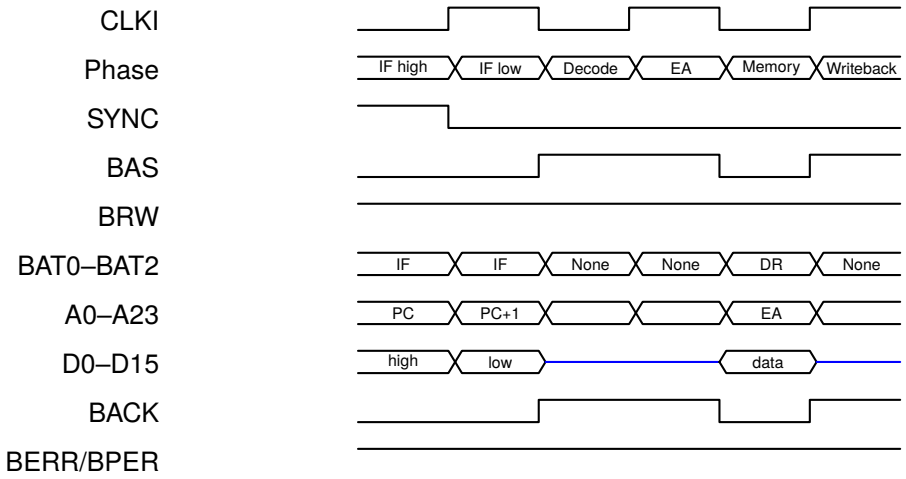


Figure 5.8: Complete LOAD instruction with no wait states

5.2 Reset and Interrupt Timing Examples

Figure 5.9 shows a hardware reset where RSTI is asserted for one clock. Holding RSTI asserted for longer extends the reset-output interval; the reset-vector fetch does not begin until RSTI is deasserted and the reset-output hold has completed.

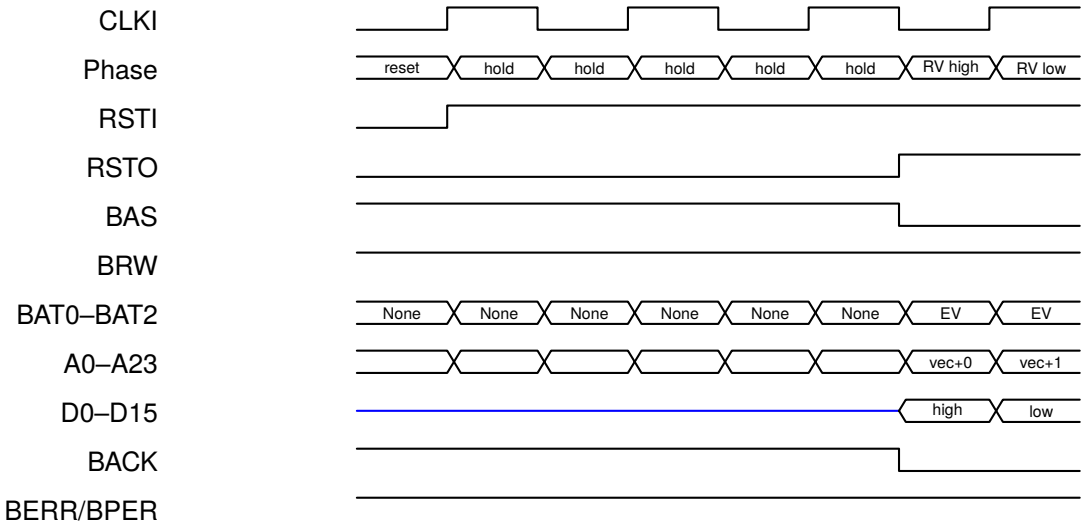


Figure 5.9: Hardware reset with one-cycle RSTI assertion and reset-vector fetch

Figure 5.10 shows an enabled maskable interrupt asserted before an instruction boundary. The CPU samples the interrupt lines on every clock cycle, so the request is latched in the cycle it is first seen, but it is dispatched only at an instruction boundary. The following exception-unit dispatch fetches the selected hardware-interrupt vector instead of fetching the next processing-unit instruction. A request latched partway through an instruction waits for the next boundary.

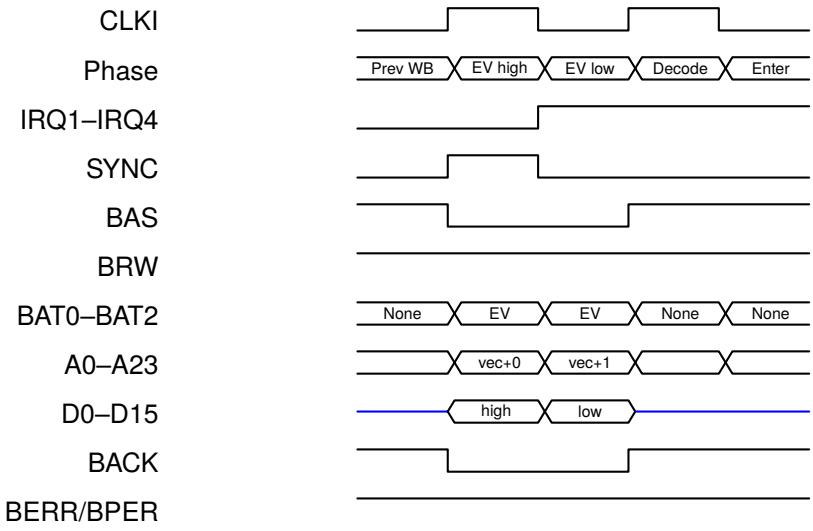


Figure 5.10: Enabled IRQ latched before an instruction boundary and dispatched with no wait states

CHAPTER 6

Electrical Characteristics

The only electrical requirement currently specified is the supply: `VCC1` and `VCC2` must be connected to 5 V $\pm$ 5% and `GND1` and `GND2` to the 0 V rail (see Chapter 3).

The bus timing in this manual is cycle-level logical timing, measured in `CLKI` cycles. The following characteristics have not yet been specified and will be added to this chapter when they are:

- Input and output voltage levels
- Clock frequency range
- Setup, hold and propagation delay times
- Fanout and maximum board loading
- Supply current and power dissipation
- Package dimensions and thermal characteristics

Until then, designs must not assume any value for these characteristics.

SIRC-1 Hardware Reference Manual

This manual describes the SIRC-1 as a physical device: the package and pinout, what every signal does, and the bus protocol the CPU speaks to memory and peripherals. It is written for the engineers who design boards, memory controllers and peripherals around the chip. What the processor does with the data once it has it is the subject of the companion SIRC-1 Architecture Reference Manual.

Inside you will find:

- The 64-pin DIP package and pin-by-pin signal descriptions
- The clocked bus protocol: transfers, wait states, bus errors and bus protection
- Cycle-level timing diagrams for instruction fetch, data access, reset and interrupts
- The rules for sampling the interrupt, halt, trace and reset inputs

SILICON INTEGRATED RESEARCH CORPORATION
1400 Fabrication Way, Milpitas Flat, CA
Printed in the United States of America

First Printing, 1989 • Version 1.0